

Bus Routes

Hard Problem

(<https://leetcode.com/problems/bus-routes/description/>)

You are given an array **routes** representing bus routes where **routes[i]** is a bus route that the **ith** bus repeats forever.

For example, if **routes[0] = [1, 5, 7]**, this means that the **0th** bus travels in the sequence **1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ... forever**.

You will start at the bus stop **source** (You are not on any bus initially), and you want to go to the bus stop **target**. You can travel between bus stops by buses only.

Input

The first input line has one integer **n**: the number of bus routes.

After that, there are **n** lines that describe the **nth** bus stopped. Each line starts with an integer **k**, followed by **k** distinct integers **b**, representing the stops on that bus route.

The last line contains two integers **x** and **y**: the starting and destination bus stop.

Output

Print the **least number of buses** you must take to travel from **source** to **target**. Print **-1** if it is not possible.

Constraints

- $1 \leq n \leq 500$
- $1 \leq k \leq 10^5$
- All the values of **b** are unique.
- $0 \leq b < 10^6$
- $0 \leq x, y < 10^6$

Examples

Example 1:

Input:

```
2
```

```
3 1 2 7
3 3 6 7
1 6
```

Output:

```
2
```

Explanation:

The optimal strategy is:

1. Take the first bus (route [1, 2, 7]) to reach stop 7.
2. Switch to the second bus (route [3, 6, 7]) to reach stop 6.

Thus, the minimum number of buses required is 2.

Example 2:

Input:

```
5
2 7 12
3 4 5 15
1 6
2 15 19
3 9 12 13
15 12
```

Output:

```
-1
```

Explanation:

There is no route that start at 15 and end at 12.

Container With Most Water

Medium Problem

(<https://leetcode.com/problems/container-with-most-water/>)

You are given an integer array **height** of length **n**. There are **n** vertical lines drawn such that the two endpoints of the **i**th line are **(i, 0)** and **(i, height[i])**.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Notice that you may not slant the container.

Input

The first input line contains an integer **n**: the number of vertical lines.

The next line has **n** integers **x₁, x₂, ..., x_n**: the height of the **i-th** vertical line.

Output

Print the **maximum** amount of water a container can store.

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq x_i \leq 10^4$

Example

Example 1:

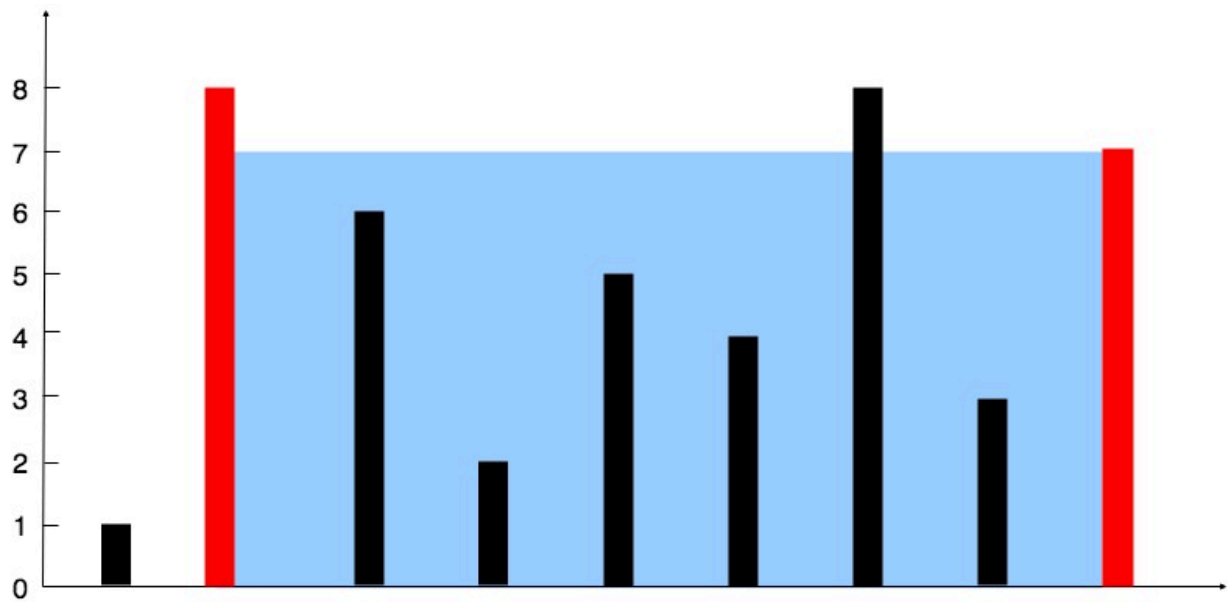
Input:

```
9
1 8 6 2 5 4 8 3 7
```

Output:

```
49
```

Explanation:



The above vertical lines are represented by an array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input:

```
2
1 1
```

Output:

```
1
```

Robot Return to Origin

Easy Problem (<https://leetcode.com/problems/robot-return-to-origin/>)

There is a robot starting at the position **(0, 0)**, the origin, on a 2D plane. Given a sequence of its moves, judge if this robot **ends up at (0, 0)** after it completes its moves.

You are given a string **moves** that represents the move sequence of the robot where **moves[i]** represents its **ith** move. Valid moves are **'R'** (right), **'L'** (left), **'U'** (up), and **'D'** (down).

Note: The way that the robot is "facing" is irrelevant. **'R'** will always make the robot move to the right once, **'L'** will always make it move left, etc. Also, assume that the magnitude of the robot's movement is the same for each move.

Input

A string moves consisting of characters **'R'**, **'L'**, **'U'**, and **'D'**.

Output

Print **true** if the robot returns to the origin after it finishes all of its moves, or **false** otherwise.

Constraints

- $1 \leq \text{moves.length} \leq 2 * 10^4$
- **moves** only contains the characters **'U'**, **'D'**, **'L'** and **'R'**.

Examples

Example 1:

Input:

```
UD
```

Output:

```
true
```

Explanation:

```
The robot moves up once, and then down once. All moves have the same magnitude, so it ended up at the origin where it started. Therefore, we return true.
```

Example 2:

Input:

```
LL
```

Output:

```
false
```

Explanation:

```
The robot moves left twice. It ends up two "moves" to the left of the
origin. We return false because it is not at the origin at the end of its
moves.
```